

Nile Basin Explorer Project Report

By: Mohamed Tarek Mohamed

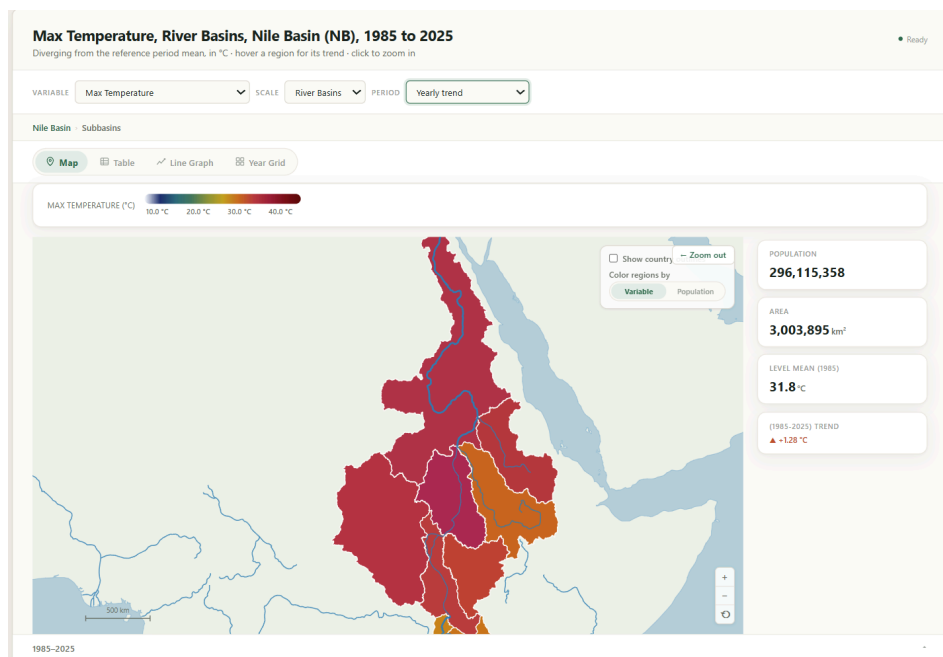
Date: 03/09/2026

Project URL: [nile-viz-complete](#)

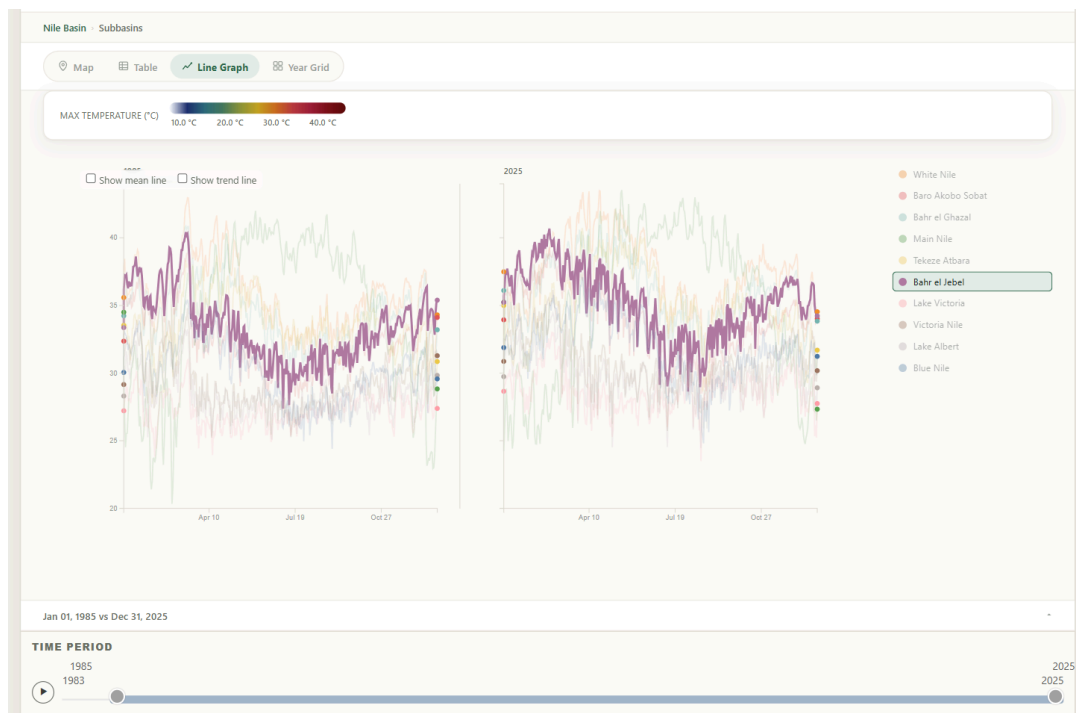
1. Current Overview

This summer project aimed to build an interactive tool for exploring the climate of the Nile Basin — the geographic area surrounding the Nile River — drawing on 42 years of climate data, including drought levels, precipitation, number of dry days, and maximum temperature.

The resulting explorer presents this data through four coordinated views: a map, a table, a line chart, and a year grid. From any of these views, a user can drill down from the whole basin to an individual country and then to a subregion, switch between yearly and daily granularity, and step through a daily animation (12 fps, with a hover-preview and a fullscreen mode) of how conditions evolve over time. The interface also supports direct comparison: a user can select any two years, or any two days, and see how a chosen variable's value differed between them side by side.



The most useful outcome of the tool is being able to see, at a glance, how the basin's climate has changed over time or how different areas compare to one another. For example, a user investigating a known drought year could select a specific subregion, compare its precipitation for that year against the surrounding years, and see immediately how far below average conditions fell — or compare two countries in the same year to see which received more rainfall and by how much.



2. Data and Processing

The system is built around three parts that work together: **data loading and storage**, **user-interface controls**, and **visualization**.

Data loading and storage. Before the data ever reaches the app, a companion Python pipeline (`extract_hierarchical.py` , now past its 12th revision) processes the raw climate source files — ERA5 and CID NetCDF datasets — into a partitioned Parquet layout. This step handles the differences between source datasets, including multiple variables stored in a single file, yearly versus daily granularity, and differing grid shapes across the two sources. Inside the app itself, this Parquet data is loaded directly in the browser using DuckDB-WASM — a version of the DuckDB database compiled to run client-side — through methods like `loadDataset` and `loadYearlyAggregates` . Each dataset is queried once and cached in a shared

`state.datasets` object, so switching views or drilling down does not re-trigger the same query.

User-interface controls. A set of shared controls sits above the four views: a drill-down breadcrumb (basin → country → subregion), a variable picker, a timeline slider with a draggable range bar, and a compare-mode toggle. None of these controls talk to a specific view directly — instead, any change a user makes is written into one shared, global state object.

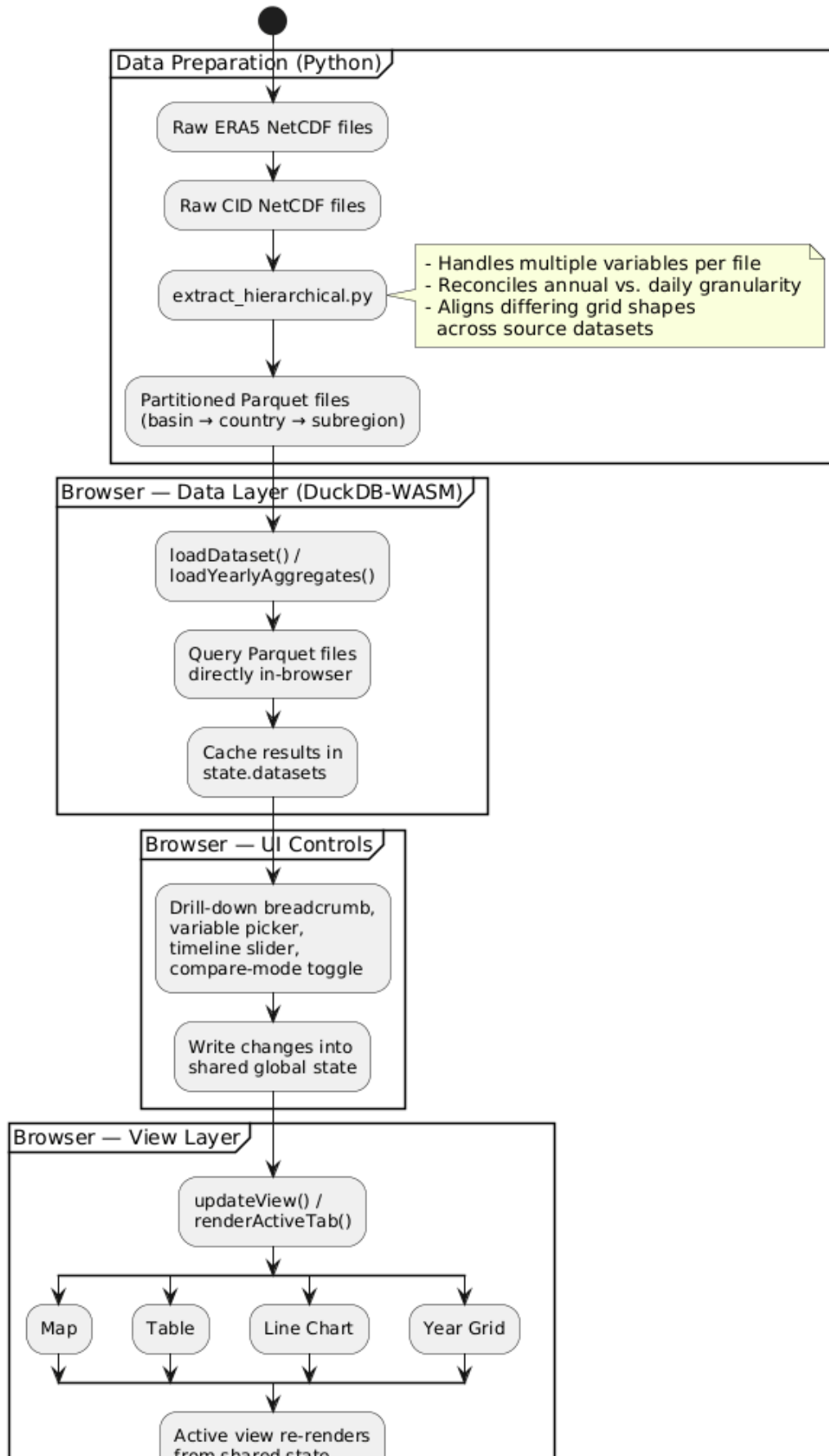
Visualization. The map, table, line chart, and year grid are each driven by their own rendering function, and each one reads from that same shared state, triggered through `updateView()` or `renderActiveTab()`. When a control changes — for example, the user drills into a new subregion or drags the timeline — the state update triggers a re-render of whichever view is currently active, so the map, table, chart, and grid always stay in sync without needing to know about one another. Dynamic, per-variable color scales (including purpose-built schemes for drought severity, tropical precipitation bursts, and NWS-style temperature banding) are computed alongside a separate categorical colour scale that keeps each region's identity consistent across views, independent of whatever gradient the map is currently showing.

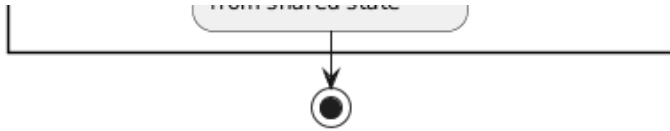
Put together, the flow runs from the raw NetCDF files, through the extraction pipeline into partitioned Parquet files, into DuckDB-WASM queries in the browser, into the shared state object, and finally out to whichever of the four views is on screen.

The project's design deliberately follows the style of Our World in Data (OWID), a widely used reference for public data visualization. OWID's charts require no server: each chart is a self-contained, interactive piece of front-end code that queries and renders its own data directly in the visitor's browser, with no backend database or API call involved. This project follows that same model — describing the app as "entirely front-end" means there is no server-side component at all; all data querying, filtering, and rendering happens locally in the browser using DuckDB-WASM and D3.js. In practice, this also means the project has no backend to host or maintain: the app is three files — `index.html` (page structure and layout), `styles.css` (all visual styling), and `app.js` (application logic) — that can be run by starting a static file server such as `python -m http.server 8080` and opening it in a browser, with no build step or dependency installation required. Functions

are fully documented with comments, and there is a top-level explanation of the architecture at the head of `app.js` .

Nile Basin Explorer — Data Processing Pipeline





Resources

- [D3.js documentation](#) — the core library the whole front end is built on; the [D3 gallery](#) is useful for finding a working example close to a given chart type before writing one from scratch.
- [DuckDB-WASM documentation](#) — covers instantiation (CDN vs. bundler), querying, and reading Parquet/CSV/JSON directly in the browser; directly relevant to the `loadDataset` / `loadYearlyAggregates` data layer.
- [Apache Parquet documentation](#) — background on the file format itself (columnar layout, row groups, metadata), useful when deciding how to partition new data for the extraction pipeline.
- [Our World in Data](#) — the design reference for this project's chart style; worth revisiting directly for layout and interaction ideas as the UI/UX work continues.
- [FastAPI documentation](#) — the leading candidate framework for the planned backend.
- [Nile Basin Initiative](#) — the main intergovernmental body for the basin; a starting point for the agency-collaboration work above, and a reference for what basin-wide monitoring systems already exist.